Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

RAPPORT PROJET
Cycle Ingénieur 4ème Année

Cybersécurité

Réalisé par

Manal Elhani – Wissal Elbekkali – Zineb Elhadri – Ayman Bouissehak

BRUTE FORCE ATTACK

Encadré par : Othmane Cherqi
Présenté devant le jury : Othmane Cherqi & Harazeem Abdulganiyu

2025 -2026

Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

Étude et mise en œuvre d'une attaque par force brute sur une application web et analyse des contre-mesures de sécurité

Ce projet a pour objectif de sensibiliser aux risques de sécurité liés aux mots de passe faibles et de démontrer l'importance des mécanismes de protection contre les attaques par force brute, dans un cadre strictement éducatif.

Résumé :

Ce projet porte sur l'étude et la démonstration des attaques par force brute ciblant les mécanismes d'authentification des applications web. Une application web volontairement vulnérable a été conçue afin de reproduire un scénario réaliste d'attaque par force brute sur un formulaire de connexion. L'attaque a été mise en œuvre à l'aide d'outils automatisés reconnus dans le domaine de la cybersécurité, permettant d'illustrer l'impact des mots de passe faibles et de l'absence de mécanismes de protection.
Dans un second temps, plusieurs contre-mesures de sécurité ont été implémentées et analysées, notamment la limitation du nombre de tentatives et l'utilisation de CAPTCHA. Une comparaison des résultats avant et après l'application de ces mécanismes a permis de mettre en évidence leur efficacité dans la protection des applications web contre les attaques par force brute. Ce travail s'inscrit dans une démarche pédagogique et éthique visant à sensibiliser aux bonnes pratiques de sécurisation des systèmes d'authentification.

Mots clés :

Attaque par force brute, authentification web, cybersécurité, mots de passe faibles, Hydra, CAPTCHA, authentification multi-facteurs

Sommaire

I- Introduction *6*

1- Contexte et problématique 6

2- Objectifs du projet 8

3- Délimitation du projet 9

II- Fondements théoriques *10*

1. Les attaques par force brute 10
   1.1 Définition 10
   1.2. Différents types d'attaques par force brute 12
2. Facteurs influençant le succès des attaques par force brute 14
   2.1. Complexité et longueur du mot de passe 15
   2.2. Comportement des utilisateurs et mauvaises pratiques 15
   2.3. Absence ou insuffisance des mécanismes de protection côté serveur 16
   2.4. Attaques en ligne et attaques hors ligne 16
3. Les mots de passe faibles : risques et statistiques 17
   3.1. Statistiques récentes (2024–2025) 17
   3.2. Temps de craquage selon la longueur et la complexité 18
   3.3. Conséquences d'une compromission 19

III- Mise en œuvre : Application web vulnérable *20*

1. Structure du projet 20
   1.1. Choix des technologies 20
   1.2. Conception du formulaire de connexion 21
2. Base de données 22
3. Diagrammes de l'application 22

IV- Démonstration de l'attaque par force brute *23*

1. Présentation des outils utilisés 23
   1.1. Outil 1 : Hydra 24
   1.2. Outil 2 : Burp Suite 24
2. Liste de mots de passe utilisée 25
3. Scénario d'attaque par force brute avant et après protection 26
4. Comparaison entre l'application non sécurisée et l'application sécurisée 27

V- Mesures de protection contre les attaques par force brute *28*

1. Mesures côté serveur 28
   1.1. Limitation du nombre de tentatives et verrouillage de compte 28
   1.2. CAPTCHA 29
   1.3. Authentification multi-facteurs (2FA / MFA) 29
2. Mesures côté client 30
3. Bonnes pratiques de gestion des mots de passe 30

VI- Conclusion *30*

VII- Annexes 32

1. Code PHP 32

Figure 1 : Initialisation de la session et mécanisme de limitation des tentatives de connexion 32
Figure 2 : Vérification CAPTCHA et authentification sécurisée avec base de données 33
Figure 3 : Gestion des échecs de connexion, verrouillage du compte et déconnexion utilisateur 33

2. Login page avant sécurité 34

Figure 4 : Interface de connexion sans mécanismes de sécurité 34
Figure 5 : Connexion réussie sur une application non protégée 34

3. Login page après sécurité 35

Figure 6 : Interface de connexion avec CAPTCHA et limitation des tentatives 35

4. Outils d'attaque et d'analyse utilisés 36

Figure 8 : Burp Suite Intruder avant sécurité 36
Figure 9 : Hydra avant sécurité 36
Figure 10 : Burp Suite après sécurité 37
Figure 11 : Hydra après sécurité 37

Diagramme 1 : Diagramme de classe de l'application..... 23
Diagramme 2 : Diagramme de séquence - Attaque Brute Force (sans protection) .. 26
Diagramme 3 : Diagramme de séquence - Attaque Brute Force (avec protection) .. 27

I- Introduction

1- Contexte et problématique

Dans un contexte marqué par une transformation numérique rapide et globale, les applications web occupent aujourd'hui une place centrale dans la fourniture de services numériques. Elles sont devenues le principal moyen d'accès à des plateformes essentielles telles que les services bancaires en ligne, le commerce électronique, les réseaux sociaux, les systèmes de messagerie, les plateformes éducatives ou encore les services administratifs. Cette omniprésence des applications web s'accompagne d'une manipulation croissante de données sensibles, personnelles et parfois critiques, ce qui en fait des cibles privilégiées pour les cyberattaques.

L'un des éléments fondamentaux de la sécurité des applications web est le mécanisme d'authentification des utilisateurs. Dans la majorité des cas, ce mécanisme repose sur un modèle simple et largement répandu : l'utilisation d'un couple identifiant/mot de passe. Ce modèle, apprécié pour sa simplicité de mise en œuvre et sa facilité d'utilisation, reste aujourd'hui le plus utilisé à travers le monde. Toutefois, cette simplicité constitue également sa principale faiblesse. En effet, lorsqu'il est mal implémenté ou insuffisamment protégé, le mécanisme d'authentification devient un point d'entrée critique pour les attaquants.

Parmi les nombreuses menaces ciblant les systèmes d'authentification, l'attaque par force brute (brute force attack) figure parmi les techniques les plus anciennes, mais également parmi les plus efficaces dans certains contextes. Cette attaque consiste à tester de manière systématique un grand nombre de combinaisons possibles de mots de passe jusqu'à identifier la combinaison correcte. Bien qu'elle repose sur un principe relativement simple, la force brute tire son efficacité de l'automatisation et de la puissance de calcul disponible, plutôt que de l'exploitation de failles logicielles complexes.

Contrairement à d'autres types d'attaques plus sophistiquées, telles que les injections SQL ou les attaques par cross-site scripting (XSS), l'attaque par force brute ne nécessite pas de connaissance approfondie du code source de l'application ciblée. Elle exploite uniquement le comportement observable du système face à des tentatives répétées d'authentification. Cette caractéristique rend ce type d'attaque particulièrement accessible, même à des attaquants peu expérimentés, grâce à la disponibilité d'outils automatisés largement diffusés sur Internet.

L'évolution constante des technologies matérielles a considérablement renforcé l'efficacité des attaques par force brute. Les processeurs multicœurs, les cartes graphiques (GPU) capables d'effectuer des calculs massivement parallèles, ainsi que l'accès à des ressources de calcul via des services cloud, ont drastiquement réduit le temps nécessaire pour tester un grand nombre de combinaisons. Par ailleurs, l'existence de bases de données de mots de passe issues de fuites massives (data breaches) a favorisé l'émergence de listes de mots de passe couramment utilisés, augmentant encore les chances de succès de ce type d'attaque lorsque les utilisateurs adoptent de mauvaises pratiques.

De nombreuses études et rapports en cybersécurité indiquent que les attaques par force brute figurent régulièrement parmi les vecteurs d'intrusion les plus exploités, en particulier sur les formulaires de connexion exposés publiquement. Elles sont souvent utilisées comme première étape d'une compromission plus large, permettant ensuite l'accès à des données sensibles, l'escalade de privilèges ou le déploiement d'attaques secondaires. Cette persistance des attaques par force brute souligne le fait que de nombreuses applications continuent d'implémenter des mécanismes d'authentification insuffisamment sécurisés.

Face à cette réalité, il devient indispensable de comprendre non seulement le fonctionnement de ces attaques, mais également les raisons pour lesquelles elles réussissent encore aujourd'hui. Une mauvaise gestion des mots de passe, l'absence de limitation du nombre de tentatives, le manque de mécanismes de détection ou encore l'absence d'authentification renforcée constituent autant de facteurs favorisant leur succès. À l'inverse, l'intégration de contre-mesures adaptées peut considérablement réduire, voire neutraliser, l'impact de ce type d'attaque.

La problématique centrale de ce projet s'articule donc autour de la question suivante : comment une attaque par force brute peut-elle compromettre la sécurité d'une application web classique, et quelles sont les mesures efficaces permettant de s'en prémunir ? Pour répondre à cette problématique, ce travail propose une approche expérimentale et pédagogique consistant à concevoir une application web volontairement vulnérable, à y simuler une attaque par force brute, puis à analyser l'impact de différentes contre-mesures de sécurité.

Ce projet s'inscrit dans une démarche strictement éthique et académique. L'objectif n'est en aucun cas de promouvoir des pratiques malveillantes, mais de sensibiliser aux risques réels liés à une mauvaise conception des mécanismes d'authentification. En mettant en évidence les failles les plus courantes et en démontrant l'efficacité de solutions simples et accessibles, ce travail vise à renforcer la compréhension des enjeux de la sécurité des applications web et à

encourager l'adoption de bonnes pratiques dès la phase de conception. **Dans quelle mesure une attaque par force brute peut-elle compromettre la sécurité d'une application web classique, et comment l'implémentation de contre-mesures adaptées permet-elle d'en limiter efficacement l'impact ?**

2- Objectifs du projet

L'objectif principal de ce projet est de démontrer de manière concrète et expérimentale le fonctionnement d'une attaque par force brute sur une application web volontairement vulnérable, afin de mettre en évidence les risques liés à une mauvaise conception des mécanismes d'authentification. À travers cette démarche, le projet vise également à analyser l'efficacité de différentes contre-mesures de sécurité couramment utilisées pour limiter ou empêcher ce type d'attaque.

Dans cette optique, le projet s'appuie sur une approche progressive, allant de la conception d'un système vulnérable à l'implémentation de solutions de protection, en passant par l'observation et l'analyse des résultats obtenus. Cette méthodologie permet de comprendre non seulement *comment* une attaque par force brute est réalisée, mais surtout *pourquoi* elle réussit et *comment* elle peut être efficacement contrée.

Les objectifs spécifiques de ce projet sont les suivants :

- Concevoir et développer une application web simple intégrant un mécanisme d'authentification basé sur un formulaire de connexion relié à une base de données, afin de reproduire un scénario réaliste et représentatif des applications web courantes.
- Mettre en œuvre une attaque par force brute à l'aide d'outils ou de scripts automatisés capables de tester un grand nombre de mots de passe de manière répétitive, dans le but d'évaluer la vulnérabilité du mécanisme d'authentification initial.
- Analyser les facteurs influençant la rapidité et le taux de succès de l'attaque, notamment la complexité et la longueur des mots de passe, ainsi que l'absence ou la présence de mécanismes de protection côté serveur.
- Implémenter et évaluer l'efficacité de plusieurs contre-mesures de sécurité largement utilisées dans les applications web modernes, telles que la limitation du nombre de tentatives de connexion, l'introduction de délais entre les tentatives et l'utilisation de CAPTCHA.

- Comparer les résultats obtenus avant et après la mise en place des contre-mesures afin de mesurer leur impact réel sur la protection contre les attaques par force brute.
- Sensibiliser aux bonnes pratiques de sécurisation des mécanismes d'authentification, tant du point de vue du développement que de celui des utilisateurs, en mettant en évidence l'importance des mots de passe robustes et des mécanismes de sécurité complémentaires.

Enfin, ce projet possède une vocation essentiellement pédagogique et académique. Il ne vise en aucun cas à attaquer des systèmes réels ou des services en production, mais à illustrer, dans un environnement contrôlé et éthique, les risques associés aux attaques par force brute ainsi que les solutions permettant de renforcer la sécurité des applications web.

3- Délimitation du projet

Afin de garantir la faisabilité du projet dans le temps imparti et de maintenir un cadre pédagogique, réaliste et maîtrisé, plusieurs délimitations ont été définies dès le début du travail. Ces choix permettent de se concentrer sur les aspects essentiels liés aux attaques par force brute et à leur mitigation, sans introduire une complexité excessive susceptible de nuire à la clarté de l'analyse.

Tout d'abord, l'application web développée dans le cadre de ce projet est volontairement simplifiée. Elle se limite à un mécanisme d'authentification basique comprenant un seul formulaire de connexion relié à une base de données contenant un nombre restreint de comptes de test. Aucune fonctionnalité avancée n'a été implémentée, telles que la gestion de sessions complexes, les rôles utilisateurs, le chiffrement côté client ou les mécanismes de récupération de mot de passe. Ce choix vise à isoler le mécanisme d'authentification afin d'en analyser précisément les vulnérabilités face aux attaques par force brute.

Concernant la phase d'attaque, celle-ci est réalisée exclusivement dans un environnement contrôlé, en local et sur un réseau privé. Aucune tentative d'attaque n'a été effectuée sur des services tiers, des plateformes publiques ou des systèmes en production. Cette restriction garantit le respect des principes éthiques et légaux encadrant les travaux en cybersécurité, tout en assurant un environnement sûr pour l'expérimentation.

Les mots de passe utilisés lors de la démonstration de l'attaque sont issus de listes publiques couramment utilisées dans un contexte pédagogique, telles que des listes réduites inspirées de fuites connues comme RockYou, ou générés de manière simple. L'objectif n'est pas

d'optimiser l'attaque au maximum à l'aide de techniques avancées, telles que l'utilisation de cartes graphiques (GPU), de clusters de calcul ou d'attaques distribuées, mais de démontrer l'efficacité d'une attaque basique face à des mots de passe faibles.

En ce qui concerne les contre-mesures, le projet se concentre volontairement sur des mécanismes de protection classiques et largement répandus dans les applications web. Parmi ceux-ci figurent la limitation du nombre de tentatives de connexion, l'introduction de délais entre les tentatives et l'utilisation de CAPTCHA simples. Des solutions plus complexes ou spécifiques à des environnements industriels n'ont pas été abordées afin de préserver la lisibilité et la reproductibilité du projet.

Enfin, le périmètre de l'étude exclut volontairement certaines variantes avancées des attaques par force brute, telles que les attaques hybrides sophistiquées, le credential stuffing, les attaques par reverse brute force ou les scénarios impliquant des botnets à grande échelle. Ces techniques, bien que pertinentes dans un contexte professionnel, dépassent le cadre pédagogique de ce projet et nécessiteraient des infrastructures, des jeux de données et des autorisations spécifiques.

Ces délimitations permettent ainsi de proposer une étude ciblée, cohérente et éthique, tout en offrant une compréhension claire des risques liés aux attaques par force brute et des solutions permettant de s'en protéger efficacement.

II- Fondements théoriques

1. Les attaques par force brute

1.1 Définition

Une attaque par force brute (brute force attack) est une technique d'attaque informatique consistant à tester de manière systématique et automatisée un grand nombre de combinaisons possibles afin de découvrir un secret protégé, tel qu'un mot de passe, une clé de chiffrement ou des identifiants d'authentification. Contrairement à d'autres attaques reposant sur l'exploitation de failles logicielles spécifiques ou d'erreurs de configuration, la force brute s'appuie principalement sur la répétition d'essais successifs et sur la puissance de calcul disponible.

Le principe fondamental de cette attaque repose sur l'hypothèse qu'un secret suffisamment court ou prévisible finira nécessairement par être découvert si toutes les combinaisons possibles

sont testées. Cette approche peut être exhaustive, lorsque l'ensemble de l'espace de recherche est parcouru, ou partielle, lorsque l'attaquant restreint volontairement les combinaisons testées en se basant sur des hypothèses liées aux habitudes humaines ou aux contraintes du système ciblé.

Selon l'Open Web Application Security Project (OWASP), une attaque par force brute consiste généralement à définir un ensemble de valeurs prédéterminées, à envoyer des requêtes répétées à un service d'authentification et à analyser les réponses retournées par le serveur afin d'identifier une combinaison valide. Cette analyse peut s'appuyer sur différents indicateurs, tels que le contenu du message de réponse, le code HTTP, la longueur de la réponse ou encore le temps de traitement. L'attaque est le plus souvent automatisée à l'aide de scripts ou d'outils spécialisés, ce qui permet d'atteindre un volume de tentatives très élevé en un temps réduit.

Dans le contexte des applications web, l'attaque par force brute cible principalement les formulaires de connexion accessibles publiquement. Chaque tentative correspond à une requête HTTP ou HTTPS envoyée au serveur, simulant une tentative de connexion légitime. En l'absence de mécanismes de protection adaptés, le serveur traite ces requêtes sans distinction entre un utilisateur humain et un script automatisé, rendant l'attaque difficile à détecter dans ses phases initiales.

L'efficacité d'une attaque par force brute dépend directement de plusieurs facteurs, notamment la longueur du mot de passe, la diversité des caractères utilisés et la présence de mécanismes de sécurité côté serveur. Le nombre total de combinaisons possibles augmente de manière exponentielle avec la longueur du mot de passe et la taille de l'alphabet utilisé. Par exemple, un mot de passe de 8 caractères composé uniquement de lettres minuscules offre 26^8 combinaisons possibles, tandis qu'un mot de passe alphanumérique incluant majuscules et chiffres dépasse les 10^{14} possibilités. Bien que cet espace de recherche soit théoriquement très vaste, il peut être considérablement réduit dans la pratique lorsque les utilisateurs choisissent des mots de passe faibles ou prévisibles.

Les avancées technologiques ont par ailleurs renforcé la viabilité de ce type d'attaque. L'utilisation de processeurs multicœurs, de cartes graphiques capables d'exécuter des calculs parallèles à grande échelle (GPU) et de services de calcul à la demande via le cloud permet aujourd'hui de tester des millions, voire des milliards de combinaisons par seconde dans certains contextes, notamment lors d'attaques hors ligne. Dans le cas des attaques en ligne, bien que le débit soit limité par le temps de réponse du serveur et la bande passante, l'automatisation permet néanmoins d'atteindre des volumes significatifs de tentatives.

Il convient également de distinguer les attaques par force brute menées en ligne de celles menées hors ligne. Dans une attaque en ligne, l'attaquant interagit directement avec le service d'authentification, ce qui rend l'attaque plus facilement détectable mais également plus simple à mettre en œuvre. À l'inverse, une attaque hors ligne s'appuie sur des données préalablement récupérées, telles que des bases de mots de passe hachés issues de fuites, et permet d'exploiter pleinement la puissance de calcul disponible sans limitation imposée par un serveur distant.

Enfin, bien que l'attaque par force brute soit souvent perçue comme une méthode rudimentaire, elle demeure une menace sérieuse dans de nombreux environnements réels. Son efficacité repose moins sur sa sophistication technique que sur la persistance de mauvaises pratiques en matière de gestion des mots de passe et sur l'absence de mécanismes de défense élémentaires. C'est précisément pour cette raison qu'elle constitue un sujet d'étude pertinent dans le cadre de la sécurité des applications web.

1.2. Différents types d'attaques par force brute

Les attaques par force brute ne se limitent pas à une approche unique et uniforme. En pratique, plusieurs variantes existent, chacune étant adaptée en fonction des ressources techniques disponibles à l'attaquant, du temps dont il dispose et, surtout, des comportements habituels des utilisateurs en matière de choix de mots de passe. Ces variantes reposent toutes sur le même principe fondamental — la répétition de tentatives d'authentification — mais diffèrent par leur stratégie et leur efficacité.

Les utilisateurs ayant tendance à choisir des mots de passe simples, courts ou basés sur des informations personnelles, les attaquants exploitent ces habitudes pour optimiser leurs attaques. Les principales variantes de la force brute sont présentées ci-dessous.

- ***Brute force pure***

La brute force pure, également appelée brute force simple, est la forme la plus basique et la plus théorique de l'attaque par force brute. Elle consiste à tester systématiquement l'ensemble des combinaisons possibles de caractères sans aucune hypothèse préalable sur le mot de passe ciblé. Les combinaisons sont généralement testées dans un ordre déterminé, souvent en commençant par les mots de passe les plus courts avant d'augmenter progressivement la longueur.

Par exemple, dans le cas d'un mot de passe composé uniquement de chiffres et de longueur fixe égale à quatre caractères, l'attaquant testera successivement les combinaisons allant de « 0000 » à « 9999 ». Cette approche garantit mathématiquement la découverte du mot de passe si celui-ci appartient à l'espace de recherche défini et si le temps ainsi que les ressources nécessaires sont suffisants.

L'un des principaux avantages de la brute force pure réside dans son caractère exhaustif. En théorie, elle permet de découvrir n'importe quel mot de passe, indépendamment de sa complexité ou de son originalité. Toutefois, cet avantage s'accompagne d'un inconvénient majeur : le coût temporel. Le nombre de combinaisons à tester augmente de manière exponentielle avec la longueur du mot de passe et la diversité des caractères utilisés, rendant cette méthode rapidement impraticable face à des mots de passe longs ou complexes.

En pratique, la brute force pure est rarement utilisée seule dans des scénarios réels, notamment contre des systèmes correctement protégés. Elle demeure néanmoins pertinente d'un point de vue théorique et pédagogique, car elle permet d'illustrer clairement la notion d'espace de recherche et l'importance de la longueur des mots de passe. Cette variante est qualifiée de « pure » car elle ne repose sur aucune hypothèse liée aux comportements humains ou aux habitudes des utilisateurs.

- ***Attaque par dictionnaire***

L'attaque par dictionnaire constitue une variante plus pragmatique et nettement plus efficace que la brute force pure dans de nombreux contextes réels. Elle repose sur l'utilisation d'une liste prédéfinie de mots de passe potentiels, appelée dictionnaire, contenant des mots courants, des mots de passe fréquemment utilisés ou des combinaisons issues de fuites de données précédentes. Des listes célèbres, telles que RockYou.txt, contiennent plusieurs millions d'entrées et sont largement utilisées dans les tests de sécurité.

Contrairement à la brute force pure, l'attaque par dictionnaire exploite directement les mauvaises pratiques des utilisateurs. De nombreux mots de passe sont basés sur des mots du langage courant, des prénoms, des noms propres, des suites de chiffres simples ou des claviers (par exemple « azerty » ou « qwerty »). L'attaquant teste ces mots tels quels ou en y appliquant

des modifications simples, comme l'ajout de chiffres à la fin ou le remplacement de certaines lettres par des symboles.

L'un des principaux avantages de cette approche est sa rapidité. En se concentrant sur un espace de recherche réduit mais pertinent, l'attaque par dictionnaire permet d'obtenir des résultats significatifs en un temps très court, en particulier lorsque les mots de passe sont faibles ou peu originaux. Des outils spécialisés tels que John the Ripper ou Hashcat proposent des modes dédiés à ce type d'attaque et permettent de gérer efficacement de grandes listes de mots de passe.

Il est important de noter que, bien que l'attaque par dictionnaire ne teste pas toutes les combinaisons possibles, elle est souvent considérée comme une forme optimisée de force brute. Son efficacité repose moins sur l'exhaustivité que sur la probabilité statistique que le mot de passe ciblé figure dans le dictionnaire utilisé.

- ***Attaque hybride***

L'attaque hybride, également appelée hybrid brute force attack, combine les principes de l'attaque par dictionnaire et de la brute force pure afin d'en tirer les avantages respectifs. Elle commence par une liste de mots de passe courants, puis applique de manière systématique des variations à chacun de ces mots à l'aide de règles ou de masques prédéfinis.

Par exemple, un mot de passe de base tel que « password » peut être décliné en de nombreuses variantes : « password1 », « password123 », « Password! », « p@ssw0rd », ou encore « password2024 ». Ces transformations sont générées automatiquement par des règles intégrées aux outils d'attaque, permettant de couvrir un large éventail de mots de passe dits « renforcés », mais construits de manière prévisible.

L'attaque hybride est particulièrement efficace contre les utilisateurs qui tentent de renforcer des mots de passe simples en y ajoutant des éléments faciles à deviner, comme une année de naissance, un chiffre unique ou un symbole placé en fin de chaîne. Elle offre un bon compromis entre vitesse et exhaustivité, tout en restant beaucoup plus réaliste que la brute force pure.

Bien que d'autres variantes existent, telles que l'attaque inversée (reverse brute force) ou le credential stuffing, celles-ci reposent sur des logiques légèrement différentes et dépassent le cadre strict de la force brute classique. Elles ne sont donc pas abordées dans ce projet, afin de conserver un périmètre clair et cohérent.

2. Facteurs influençant le succès des attaques par force brute

Le succès d'une attaque par force brute ne dépend pas uniquement de la méthode employée ou des outils utilisés par l'attaquant. Il résulte d'une combinaison de plusieurs facteurs techniques et humains liés à la conception du système d'authentification, aux comportements des utilisateurs et aux mécanismes de protection mis en place. Comprendre ces facteurs est essentiel pour évaluer le niveau de vulnérabilité d'une application web et identifier les points d'amélioration en matière de sécurité.

2.1. Complexité et longueur du mot de passe

La longueur et la complexité du mot de passe constituent les premiers éléments déterminants dans la résistance face à une attaque par force brute. D'un point de vue mathématique, le nombre de combinaisons possibles augmente de manière exponentielle avec la longueur du mot de passe et la diversité des caractères utilisés. Ainsi, un mot de passe court composé uniquement de chiffres ou de lettres minuscules présente un espace de recherche relativement limité, facilement exploitable par des outils automatisés.

À l'inverse, l'utilisation de mots de passe plus longs intégrant des majuscules, des chiffres et des caractères spéciaux augmente considérablement le nombre de combinaisons possibles, rendant l'attaque beaucoup plus coûteuse en temps et en ressources. Toutefois, dans la pratique, de nombreux utilisateurs privilégient encore des mots de passe courts ou légèrement modifiés (ajout d'un chiffre ou d'un symbole), ce qui réduit fortement l'efficacité réelle de cette complexité apparente.

Il est également important de souligner que la longueur du mot de passe joue souvent un rôle plus déterminant que la complexité seule. Un mot de passe long composé de mots aléatoires ou d'une phrase (passphrase) peut offrir une meilleure résistance qu'un mot de passe court mais complexe.

2.2. Comportement des utilisateurs et mauvaises pratiques

Le facteur humain reste l'un des maillons les plus faibles de la chaîne de sécurité. De nombreux utilisateurs adoptent des comportements qui facilitent grandement les attaques par force brute, souvent par souci de simplicité ou de mémorisation. Parmi les mauvaises pratiques les plus courantes figurent l'utilisation de mots de passe évidents, la réutilisation du même mot de passe sur plusieurs services et l'emploi d'informations personnelles facilement devinables.

Ces habitudes sont largement exploitées par les attaquants, notamment à travers les attaques par dictionnaire ou hybrides, qui reposent sur des listes de mots de passe issus de fuites de données antérieures. Lorsqu'un utilisateur réutilise un mot de passe déjà compromis sur une

autre plateforme, une attaque par force brute ou assimilée devient beaucoup plus efficace, même avec un nombre limité de tentatives.

Le manque de sensibilisation des utilisateurs aux enjeux de la sécurité contribue également à la persistance de ces vulnérabilités. En l'absence de politiques de mots de passe strictes ou de recommandations claires, les utilisateurs tendent à privilégier la commodité au détriment de la sécurité.

2.3. Absence ou insuffisance des mécanismes de protection côté serveur

Un autre facteur déterminant dans le succès des attaques par force brute est l'absence ou la mauvaise configuration des mécanismes de protection côté serveur. Lorsqu'un serveur autorise un nombre illimité de tentatives de connexion sans aucune restriction, il devient particulièrement vulnérable aux attaques automatisées.

Parmi les mécanismes de protection les plus importants figurent la limitation du nombre de tentatives, l'introduction de délais entre les tentatives successives, le verrouillage temporaire ou définitif des comptes et l'utilisation de CAPTCHA pour différencier un utilisateur humain d'un script automatisé. L'absence de journalisation (logs) ou de mécanismes de détection d'anomalies complique également l'identification des attaques en cours, permettant à celles-ci de se poursuivre sans être détectées.

De nombreuses applications vulnérables échouent à implémenter ces protections de base, soit par négligence, soit par souci de simplicité, ce qui ouvre la voie à des attaques par force brute même peu sophistiquées.

2.4. Attaques en ligne et attaques hors ligne

La distinction entre attaques par force brute en ligne et hors ligne constitue un élément clé pour comprendre leur efficacité. Une attaque en ligne implique une interaction directe avec le service d'authentification, chaque tentative étant envoyée au serveur sous forme de requête. Dans ce cas, la vitesse de l'attaque est limitée par le temps de réponse du serveur, la bande passante et les mécanismes de protection mis en place.

À l'inverse, une attaque hors ligne s'appuie sur des données préalablement obtenues, telles qu'une base de mots de passe hachés issue d'une fuite de données. Dans ce scénario, l'attaquant peut tester un nombre massif de combinaisons sans aucune limitation imposée par un serveur distant, exploitant pleinement la puissance de calcul disponible. Les attaques hors ligne sont donc particulièrement dangereuses lorsque les mots de passe sont mal hachés ou stockés en clair.

Cette distinction souligne l'importance non seulement de protéger les mécanismes d'authentification, mais également de sécuriser le stockage des mots de passe afin de limiter l'impact d'une éventuelle compromission.

3. Les mots de passe faibles : risques et statistiques

Les mots de passe faibles, généralement définis comme courts, prévisibles ou réutilisés sur plusieurs services, constituent l'une des principales vulnérabilités des systèmes d'authentification modernes. Malgré les recommandations constantes des experts en cybersécurité et la multiplication des campagnes de sensibilisation, de nombreux utilisateurs continuent de privilégier la simplicité et la facilité de mémorisation au détriment de la sécurité. Cette tendance expose les systèmes informatiques à des risques accrus, en particulier face aux attaques automatisées telles que la force brute.

La persistance de ces mauvaises pratiques s'explique en grande partie par le facteur humain. Les utilisateurs perçoivent souvent les contraintes de sécurité comme une gêne plutôt que comme une nécessité, ce qui les conduit à adopter des comportements à risque, tels que l'utilisation de mots de passe simples, la réutilisation des mêmes identifiants ou encore le stockage non sécurisé de leurs informations d'authentification. Ces choix facilitent considérablement le travail des attaquants et augmentent le taux de succès des attaques par force brute et assimilées.

3.1. Statistiques récentes (2024–2025)

Les données issues des fuites de données et des rapports de cybersécurité publiés en 2024 et 2025 mettent en évidence l'ampleur persistante du problème des mots de passe faibles. Selon le rapport Verizon Data Breach Investigations Report (DBIR) 2025, environ 81 % des violations de données impliquent des mots de passe faibles, volés ou compromis, marquant une augmentation notable par rapport aux années précédentes. Cette tendance souligne l'incapacité de nombreux systèmes à imposer des politiques d'authentification suffisamment robustes.

Par ailleurs, les attaques par force brute demeurent un vecteur d'intrusion particulièrement répandu dans le domaine des applications web. En 2025, près de 37 % des attaques réussies sur les applications web ont impliqué des techniques de force brute, contre 21 % en 2024. Cette

augmentation s'explique en grande partie par la persistance de mots de passe faciles à deviner et par l'absence de mécanismes de protection efficaces sur de nombreux formulaires de connexion exposés publiquement.

Une analyse menée par Specops Software sur plus d'un milliard de mots de passe volés en 2024 révèle que 88 % des mots de passe compromis comportent moins de 12 caractères. Des séquences extrêmement courantes, telles que « 123456 », « password » ou « qwerty », continuent d'apparaître massivement, avec plus de 4,5 millions d'occurrences recensées. De plus, environ 94 % des mots de passe analysés ont été réutilisés sur au moins deux comptes différents, ce qui amplifie considérablement les risques en cas de fuite initiale.

Les statistiques montrent également un impact significatif à l'échelle individuelle. Aux États-Unis, 46 % des utilisateurs déclarent avoir vu au moins un de leurs mots de passe compromis en 2024, dont 35 % attribuent directement cet incident à l'utilisation d'un mot de passe faible. À l'échelle mondiale, les mots de passe faibles seraient responsables d'environ 30 % des violations de données, confirmant leur rôle central dans les compromissions de systèmes.

Enfin, les impacts financiers associés à ces incidents sont considérables. Le coût moyen d'une violation de données liée à des identifiants compromis s'élève à 4,88 millions de dollars en 2024, soit une augmentation de 9,7 % par rapport à 2023. Des disparités sectorielles persistent également : par exemple, le secteur des télécommunications affiche seulement 20 % de mots de passe uniques dans les breaches analysées, contre 31 % dans le secteur des ressources humaines, révélant des niveaux de maturité en sécurité variables selon les domaines d'activité.

3.2. Temps de craquage selon la longueur et la complexité

Le temps nécessaire pour craquer un mot de passe par force brute dépend principalement de deux facteurs : sa longueur et sa complexité, c'est-à-dire le nombre et le type de caractères utilisés. Cette relation est de nature exponentielle, ce qui signifie qu'une légère augmentation de la longueur du mot de passe peut entraîner une augmentation drastique du temps requis pour le compromettre.

Les estimations présentées dans ce projet reposent sur des hypothèses réalistes, incluant l'utilisation de matériel moderne tel que des ensembles de cartes graphiques haut de gamme (par exemple 12 GPU de type RTX 4090 ou RTX 5090) et l'emploi d'un algorithme de hachage robuste comme bcrypt avec un facteur de travail égal à 10. Ces conditions correspondent à des scénarios plausibles dans des environnements d'attaque avancés.

Les résultats montrent clairement que la longueur du mot de passe joue un rôle plus déterminant que la complexité seule. Par exemple, un mot de passe composé de 15 lettres minuscules générées aléatoirement peut nécessiter plusieurs centaines de millions d'années pour être craqué, tandis qu'un mot de passe de 8 caractères incluant majuscules, minuscules, chiffres et symboles peut être compromis en quelques jours seulement. Cette observation met en évidence l'efficacité des passphrases longues par rapport aux mots de passe courts artificiellement complexifiés.

En revanche, lorsque les mots de passe sont protégés par des algorithmes de hachage faibles ou obsolètes, tels que MD5, les temps de craquage chutent drastiquement. Dans certains cas, un mot de passe composé uniquement de chiffres peut être compromis en quelques secondes, illustrant l'importance cruciale du choix des mécanismes de stockage des mots de passe.

3.3. Conséquences d'une compromission

La compromission d'un mot de passe faible peut entraîner une cascade de conséquences graves, tant pour les utilisateurs individuels que pour les organisations. À l'échelle individuelle, elle expose des données sensibles telles que les comptes bancaires, les adresses électroniques, les réseaux sociaux ou les services professionnels. Ces compromissions peuvent mener à l'usurpation d'identité, au vol financier, à des campagnes de phishing ciblées ou encore à des attaques de type ransomware.

Les impacts psychologiques et financiers sur les victimes sont également significatifs. En 2024, environ 68 % des personnes touchées par une violation de données ont dû réinitialiser plusieurs comptes, parfois sur des dizaines de services différents, entraînant une perte de temps considérable et un stress durable.

Pour les entreprises, les conséquences sont tout aussi importantes. Une violation de données liée à des identifiants compromis entraîne en moyenne un coût de plusieurs millions de dollars, incluant les pertes de productivité, les coûts de remédiation, les atteintes à la réputation et les sanctions réglementaires. Dans le cadre du Règlement Général sur la Protection des Données (RGPD), les amendes peuvent atteindre jusqu'à 4 % du chiffre d'affaires annuel, représentant une menace financière majeure pour les organisations concernées.

À plus grande échelle, les mots de passe compromis servent souvent de point de départ à des attaques en chaîne, telles que le credential stuffing ou l'escalade de privilèges. Selon certaines analyses, près de 62 % des violations de données non liées à des erreurs humaines directes

résultent de l'exploitation d'identifiants volés. À long terme, ces incidents érodent la confiance des utilisateurs et soulignent l'urgence d'adopter des mécanismes de protection renforcés, tels que l'authentification multi-facteurs (MFA), l'utilisation de gestionnaires de mots de passe et la mise en œuvre de politiques de sécurité strictes.

III- Mise en œuvre : Application web vulnérable

1. Structure du projet

Dans le cadre de ce projet, une application web volontairement vulnérable a été conçue afin de permettre l'étude et la démonstration d'une attaque par force brute dans un environnement contrôlé. L'organisation générale du projet a été pensée de manière simple et structurée, dans le but de faciliter le développement, les tests, la compréhension du fonctionnement de l'application ainsi que la reproductibilité des expérimentations.

L'ensemble des fichiers de l'application est regroupé dans un dossier unique nommé **php-login-minimal.** Cette approche centralisée permet de conserver une vision globale du projet et de limiter la complexité de l'architecture, ce qui est cohérent avec l'objectif pédagogique de l'étude. L'arborescence du projet est volontairement minimale et sépare clairement les composants liés à l'interface utilisateur, à la logique serveur et à la gestion des données.

1.1. Choix des technologies

Les technologies retenues pour le développement de l'application ont été sélectionnées en fonction de leur simplicité, de leur popularité et de leur pertinence dans un contexte académique. Elles correspondent également à des technologies largement utilisées dans des applications web réelles, ce qui rend les vulnérabilités étudiées particulièrement représentatives.

- **Backend : PHP**
  PHP est utilisé pour implémenter la logique côté serveur, notamment le traitement des formulaires, la communication avec la base de données et la vérification des identifiants. Son utilisation permet de mettre en évidence des scénarios d'authentification classiques souvent rencontrés dans les applications web.
- **Base de données : MySQL**
  MySQL est employé comme système de gestion de base de données relationnelle pour

stocker les informations des utilisateurs. Son intégration avec PHP est simple et efficace, ce qui en fait un choix pertinent pour ce projet pédagogique.

- **Frontend : HTML / CSS**
  L'interface utilisateur est développée à l'aide de HTML et CSS simples. Aucun framework frontend avancé n'a été utilisé, afin de se concentrer exclusivement sur les mécanismes d'authentification et les aspects liés à la sécurité, sans introduire de complexité graphique superflue.

1.2. Conception du formulaire de connexion

Le formulaire de connexion constitue le point d'entrée principal de l'application et représente la cible directe de l'attaque par force brute. Il est conçu de manière classique, reproduisant fidèlement le fonctionnement des formulaires d'authentification présents dans de nombreuses applications web.

Le formulaire utilise la méthode **POST** pour transmettre les informations saisies par l'utilisateur au serveur. Ce choix permet d'éviter que les données sensibles, telles que le nom d'utilisateur et le mot de passe, apparaissent dans l'URL ou soient stockées dans l'historique du navigateur, assurant ainsi un niveau minimal de confidentialité côté client.

Deux champs principaux composent le formulaire :

- Un champ texte dédié à la saisie du nom d'utilisateur, obligatoire pour la soumission du formulaire.
- Un champ de type mot de passe, qui masque automatiquement les caractères saisis à l'écran afin d'empêcher leur affichage en clair.

Lors de la soumission du formulaire, les données sont envoyées au serveur, où un script backend se charge de vérifier les identifiants fournis en interrogeant la base de données. Cette vérification est effectuée de manière simple, sans mécanismes de protection avancés, ce qui rend l'application vulnérable aux attaques automatisées.

En cas d'échec de l'authentification, un message d'erreur est affiché directement sur la page, sous le formulaire. Ce message est généré dynamiquement par le serveur et reste volontairement identique, quel que soit le motif précis de l'échec. Bien que cette approche limite la divulgation d'informations sur l'existence des comptes, elle ne suffit pas à protéger l'application contre une attaque par force brute.

2. Base de données

La base de données choisie pour ce projet est **MySQL**, un système de gestion de base de données relationnelle open-source largement utilisé dans les applications web. Ce choix présente plusieurs avantages dans un contexte pédagogique et expérimental.

Tout d'abord, MySQL est reconnu pour sa robustesse et sa fiabilité dans la gestion des données d'authentification. Il offre une compatibilité étendue avec de nombreux langages de programmation, notamment PHP, ce qui facilite son intégration dans l'application développée. De plus, il permet de travailler facilement en local à l'aide d'environnements de développement tels que XAMPP, MAMP ou une installation native, rendant les tests et les démonstrations simples à mettre en œuvre.

MySQL dispose également d'outils d'administration graphiques, tels que phpMyAdmin ou MySQL Workbench, qui permettent de visualiser et de manipuler les données de manière intuitive pendant les phases de développement et de test. Enfin, son utilisation prépare à des environnements de production réels, où MySQL reste largement déployé.

La base de données contient une seule table, nommée **users**, dont la structure est volontairement minimale afin de se concentrer sur le mécanisme d'authentification. Cette table comprend :

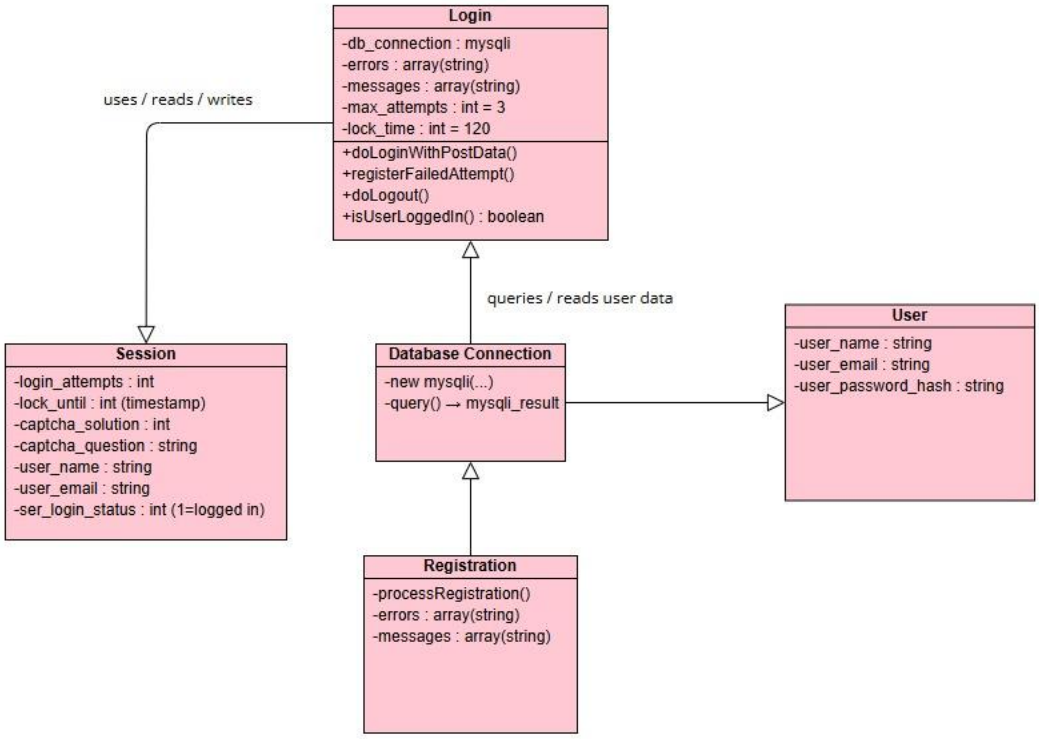
- Une clé primaire auto-incrémentée permettant d'identifier chaque utilisateur de manière unique.
- Un champ dédié au nom d'utilisateur, défini comme unique et obligatoire.
- Un champ dédié au mot de passe, également obligatoire.

Dans le cadre de cette application vulnérable, les mots de passe sont stockés en texte clair dans la base de données. Ce choix constitue une vulnérabilité intentionnelle majeure, introduite uniquement à des fins pédagogiques, afin de faciliter la démonstration des attaques par force brute et de souligner l'importance des bonnes pratiques de stockage des mots de passe, qui seront abordées ultérieurement.

3. Diagrammes de l'application

Afin de mieux comprendre le fonctionnement global de l'application et les interactions entre ses différents composants, un diagramme de classe a été réalisé. Ce diagramme illustre notamment la structure générale de l'application, le flux des données lors du processus d'authentification et la communication entre le frontend, le backend et la base de données.

Il permet de visualiser clairement le cheminement des requêtes depuis la saisie des identifiants par l'utilisateur jusqu'à la validation ou le rejet de l'authentification par le serveur. Cette représentation graphique constitue un support essentiel pour l'analyse des vulnérabilités et pour la compréhension des points d'entrée exploités lors de l'attaque par force brute.



sDiagramme 1 : Diagramme de classe de l'application

IV- Démonstration de l'attaque par force brute

1. Présentation des outils utilisés

Afin de démontrer concrètement l'efficacité d'une attaque par force brute sur l'application web vulnérable développée, deux outils largement reconnus dans le domaine de la cybersécurité ont été utilisés : **Hydra** et **Burp Suite**.

Ces outils permettent d'illustrer deux approches complémentaires : une approche automatisée en ligne de commande pour l'exécution de l'attaque, et une approche d'analyse et d'observation du trafic HTTP pour la compréhension du fonctionnement interne du formulaire de connexion.

Le choix de ces outils permet de réaliser une démonstration réaliste et professionnelle, tout en évitant le développement d'un script personnalisé, ce qui reste cohérent avec l'objectif pédagogique du projet.

1.1. Outil 1 : Hydra

Hydra est un outil open-source très populaire utilisé pour le test de résistance des mécanismes d'authentification en ligne. Il supporte un large éventail de protocoles et se révèle particulièrement efficace pour les attaques par force brute ou par dictionnaire sur des formulaires web utilisant HTTP ou HTTPS.

Dans le cadre de ce projet, Hydra a été configuré pour simuler une attaque par dictionnaire ciblant le formulaire de connexion de l'application. Plus précisément, l'outil a été utilisé pour:

- Envoyer automatiquement des requêtes HTTP POST vers l'URL du formulaire de connexion.
- Fixer un nom d'utilisateur cible, correspondant à un compte existant dans la base de données (par exemple « wissal6 »).
- Tester successivement les mots de passe contenus dans une liste prédéfinie.
- Détecter automatiquement l'échec ou la réussite d'une tentative de connexion en se basant sur le message d'erreur renvoyé par le serveur.
- Utiliser plusieurs threads afin d'accélérer le nombre de tentatives effectuées par unité de temps.

Hydra a été retenu comme outil principal pour la démonstration en raison de sa rapidité, de sa simplicité d'utilisation et de sa capacité à obtenir des résultats visibles en quelques secondes lorsque des mots de passe faibles sont utilisés. Son emploi permet de mettre en évidence la vulnérabilité de l'application face à une attaque automatisée en l'absence de mécanismes de protection.

1.2. Outil 2 : Burp Suite

Burp Suite, dans sa version Community, a été utilisé afin d'analyser le fonctionnement du formulaire de connexion et de comprendre en détail les échanges entre le navigateur et le serveur. Contrairement à Hydra, qui a été utilisé comme outil principal d'attaque par dictionnaire, Burp Suite a joué un rôle d'analyse, de préparation et de démonstration contrôlée.

Les principales actions réalisées à l'aide de Burp Suite sont les suivantes :

- Interception d'une requête de connexion légitime à l'aide du module **Proxy**, permettant d'observer la structure exacte de la requête HTTP envoyée par le navigateur.
- Identification précise des paramètres transmis au serveur, notamment les champs correspondant au nom d'utilisateur et au mot de passe.

- Analyse des en-têtes HTTP ainsi que des cookies de session échangés lors du processus d'authentification.
- Observation des différences entre une réponse d'échec et une réponse de succès, notamment en termes de contenu, de longueur de réponse et de code HTTP.
- Utilisation du module **Intruder**, de manière encadrée, afin de simuler plusieurs tentatives d'authentification automatisées et d'illustrer concrètement le principe d'une attaque par force brute.

L'utilisation d'Intruder s'est strictement limitée à une application locale volontairement vulnérable, dans un cadre pédagogique et éthique. Les informations collectées à l'aide de Burp Suite ont permis de configurer correctement l'outil Hydra, notamment en identifiant le message spécifique indiquant un échec d'authentification.

Ainsi, Burp Suite a joué un rôle essentiel dans la phase d'analyse et de compréhension du fonctionnement de l'application, tout en permettant une démonstration contrôlée du comportement du serveur face à des tentatives répétées de connexion.

2. Liste de mots de passe utilisée

Pour les démonstrations principales et les captures d'écran présentées dans ce rapport, une liste volontairement réduite de mots de passe a été utilisée. Cette liste contient uniquement neuf entrées, ce qui permet de démontrer une attaque réussie en un temps extrêmement court, rendant la démonstration particulièrement claire, impactante et facilement reproductible.

La liste de mots de passe utilisée est la suivante :

- qwertz7
- 1234
- Admin@123
- password
- 123456
- qwerty
- admin
- letmein
- Manager@123

Plusieurs de ces mots de passe correspondent directement à des mots de passe faibles insérés volontairement dans la base de données de l'application, tels que « admin », « password », «

123456 » ou « qwerty ». Les autres entrées représentent des variantes très courantes observées dans les classements annuels des mots de passe les plus utilisés.

Grâce à cette liste réduite, Hydra a systématiquement identifié les combinaisons valides en moins de cinq secondes, selon la position du mot de passe correct dans la liste. Cette rapidité illustre parfaitement la vulnérabilité de l'application face à une attaque par dictionnaire, même lorsque celle-ci est minimale.

3. Scénario d'attaque par force brute avant et après protection

Cette section présente le déroulement global du scénario de l'attaque par force brute. Elle met en relation les outils utilisés, la configuration de l'application et les résultats observés afin d'illustrer concrètement l'impact de l'absence, puis de l'ajout, de mécanismes de protection sur le succès de l'attaque. Les schémas présentés ci-dessous permettent de visualiser, dans un premier temps, le déroulement de l'attaque sur une application non sécurisée, puis, dans un second temps, le comportement de l'application après la mise en place des mesures de sécurité.

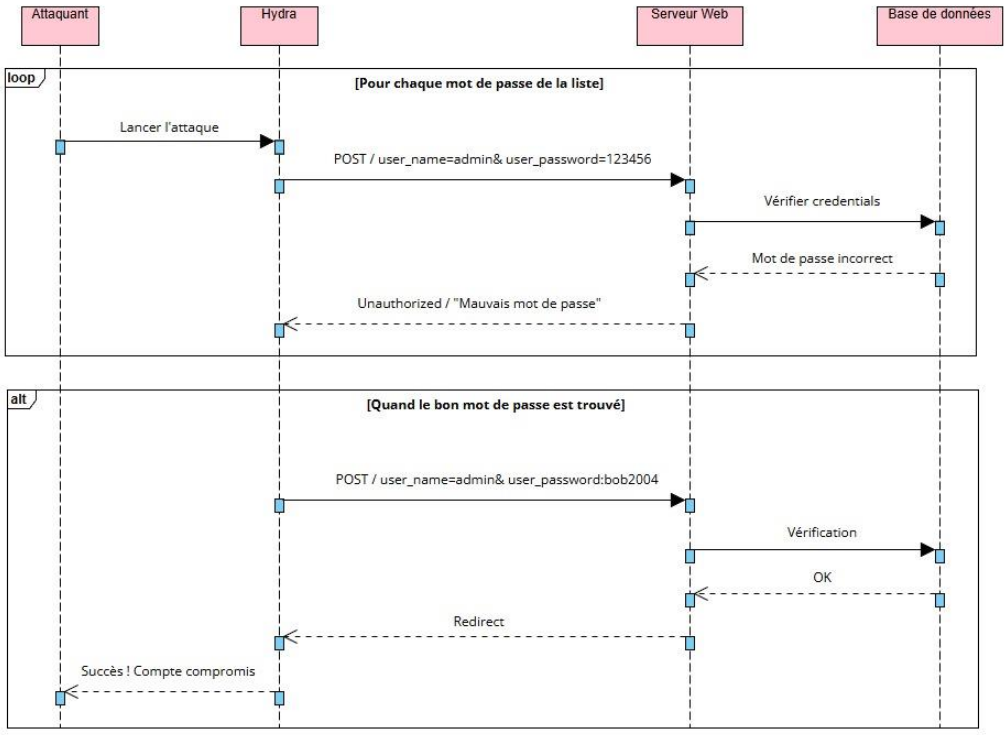


Diagramme 2 : Diagramme de séquence - Attaque Brute Force (sans protection)

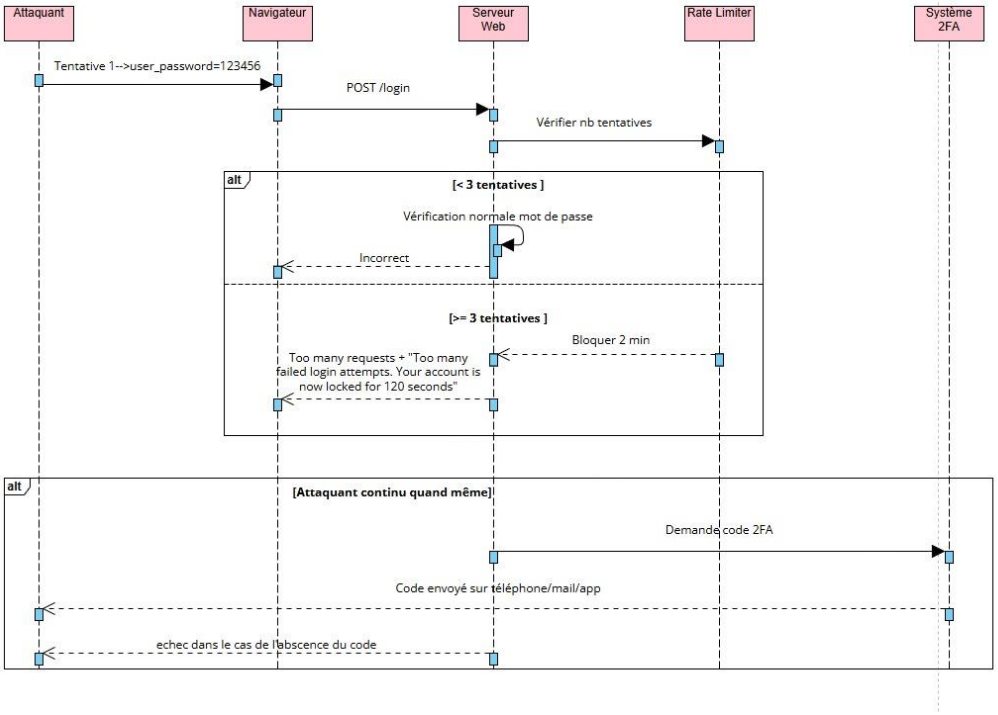


Diagramme 3 : Diagramme de séquence - Attaque Brute Force (avec protection)

4. Comparaison entre l'application non sécurisée et l'application sécurisée

Avant la mise en place des mécanismes de sécurité, l'application web présentait une vulnérabilité critique face aux attaques par force brute. En effet, le formulaire de connexion ne disposait d'aucune limitation du nombre de tentatives, ni de mécanisme de vérification permettant de distinguer un utilisateur légitime d'un script automatisé. Dans ces conditions, l'attaque par dictionnaire a pu être menée avec succès à l'aide d'outils automatisés, permettant de tester rapidement plusieurs mots de passe jusqu'à l'obtention d'un accès valide. Cette situation démontre qu'un mot de passe faible, combiné à l'absence de protections côté serveur, rend l'application facilement compromise en un temps très court.

Après l'implémentation des mécanismes de sécurité, le comportement de l'application a été significativement modifié. La limitation du nombre de tentatives de connexion à trois essais

consécutifs a permis de restreindre drastiquement la capacité d'un attaquant à multiplier les tentatives. Une fois ce seuil atteint, le compte est temporairement verrouillé, empêchant toute nouvelle tentative durant une période définie. En complément, l'ajout d'un CAPTCHA obligatoire lors de la connexion a permis de bloquer efficacement l'automatisation des requêtes, rendant l'utilisation d'outils de force brute inopérante.

La comparaison des résultats avant et après la sécurisation met clairement en évidence l'efficacité de ces contre-mesures. Alors que l'attaque aboutissait rapidement sur l'application non protégée, elle est désormais interrompue ou bloquée dès les premières tentatives sur l'application sécurisée. Cette amélioration démontre que la mise en place de mécanismes simples, tels que la limitation des tentatives et l'utilisation d'un CAPTCHA, permet de réduire considérablement les risques liés aux attaques par force brute et d'augmenter significativement le niveau de sécurité global de l'application.

V- Mesures de protection contre les attaques par force brute

Après avoir démontré l'efficacité d'une attaque par force brute sur une application web vulnérable, il est essentiel d'analyser les mécanismes de protection permettant de réduire, voire d'éliminer, ce type de menace. Les contre-mesures présentées dans cette section visent à renforcer la sécurité du mécanisme d'authentification en agissant à différents niveaux : côté serveur, côté client et au niveau des pratiques de gestion des mots de passe.

1. Mesures côté serveur

Les mesures de sécurité côté serveur constituent la ligne de défense la plus importante contre les attaques par force brute. Contrairement aux protections côté client, elles ne peuvent pas être contournées simplement par un attaquant utilisant des outils automatisés, car elles sont appliquées directement au niveau du traitement des requêtes.

1.1. Limitation du nombre de tentatives et verrouillage de compte

La limitation du nombre de tentatives de connexion consiste à restreindre le nombre d'essais autorisés pour un compte donné sur une période définie. Lorsqu'un seuil prédéfini est dépassé (par exemple 3 tentatives échouées consécutives), des mesures correctives sont appliquées, telles que le verrouillage temporaire ou définitif du compte.

Le verrouillage temporaire empêche toute nouvelle tentative pendant une durée déterminée, ce qui rend les attaques par force brute inefficaces en augmentant considérablement le temps nécessaire pour tester un grand nombre de mots de passe. Dans certains cas, un verrouillage définitif peut être appliqué, nécessitant une intervention administrative ou une procédure de réinitialisation du mot de passe.

Cette mesure présente un excellent rapport efficacité/simplicité, mais doit être implémentée avec précaution afin d'éviter les attaques par déni de service ciblant des comptes légitimes. Un compromis raisonnable consiste à combiner le verrouillage temporaire avec des mécanismes de journalisation et d'alerte.

1.2. CAPTCHA

Le CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) est un mécanisme conçu pour différencier un utilisateur humain d'un programme automatisé. Il est généralement déclenché après un certain nombre de tentatives échouées ou intégré directement au formulaire de connexion.

Dans le cadre de la protection contre les attaques par force brute, le CAPTCHA empêche efficacement l'exécution de scripts automatisés en imposant une interaction humaine difficilement reproductible par une machine. Cette mesure est particulièrement efficace contre les attaques basiques utilisant des outils automatisés.

Cependant, l'utilisation de CAPTCHA présente certaines limites, notamment en termes d'expérience utilisateur et d'accessibilité. Il est donc recommandé de l'utiliser de manière progressive ou conditionnelle, par exemple uniquement après plusieurs échecs de connexion.

1.3. Authentification multi-facteurs (2FA / MFA)

L'authentification multi-facteurs (Multi-Factor Authentication – MFA) consiste à ajouter une ou plusieurs couches de vérification supplémentaires au processus d'authentification classique. En plus du mot de passe (facteur de connaissance), l'utilisateur doit fournir un second facteur, tel qu'un code temporaire, un élément matériel ou une validation via une application dédiée.

L'intégration du 2FA permet de neutraliser efficacement les attaques par force brute, même lorsque le mot de passe est compromis. En effet, l'attaquant ne peut pas accéder au compte sans disposer du second facteur d'authentification.

Dans le cadre de ce projet, le 2FA est implémenté de manière simulée à des fins pédagogiques. Cette simulation permet de démontrer l'impact significatif de cette mesure sur la sécurité, sans nécessiter l'intégration de services externes complexes.

2. Mesures côté client

Les mesures de sécurité côté client visent principalement à améliorer l'expérience utilisateur et à décourager certaines attaques simples, bien qu'elles ne puissent pas être considérées comme des protections suffisantes à elles seules. Elles doivent toujours être combinées à des mécanismes robustes côté serveur.

Parmi ces mesures figurent la validation des champs de saisie, la désactivation temporaire du bouton de connexion après plusieurs tentatives ou l'affichage de messages d'avertissement informant l'utilisateur d'un comportement suspect. Bien que ces mécanismes puissent être contournés par un attaquant expérimenté, ils contribuent à réduire les erreurs accidentelles et à sensibiliser les utilisateurs aux enjeux de sécurité.

3. Bonnes pratiques de gestion des mots de passe

La gestion sécurisée des mots de passe constitue un élément fondamental de la protection contre les attaques par force brute. Plusieurs bonnes pratiques doivent être respectées afin de limiter les risques de compromission.

Tout d'abord, les mots de passe ne doivent jamais être stockés en clair dans la base de données. Ils doivent être protégés à l'aide d'algorithmes de hachage robustes, accompagnés d'un mécanisme de salage, afin de rendre leur exploitation difficile en cas de fuite de données. Des algorithmes tels que bcrypt, Argon2 ou PBKDF2 sont aujourd'hui recommandés.

Ensuite, la longueur du mot de passe joue un rôle essentiel dans sa résistance face aux attaques par force brute. L'utilisation de passphrases longues, composées de plusieurs mots aléatoires, offre une meilleure protection qu'un mot de passe court artificiellement complexifié.

Enfin, l'utilisation de gestionnaires de mots de passe permet aux utilisateurs de générer et de stocker des mots de passe uniques et complexes pour chaque service, réduisant ainsi les risques liés à la réutilisation des identifiants. Associée à l'authentification multi-facteurs, cette approche constitue l'une des stratégies les plus efficaces pour renforcer la sécurité des comptes utilisateurs.

VI- Conclusion

Ce projet a permis de mettre en évidence la vulnérabilité des mécanismes d'authentification lorsqu'ils reposent uniquement sur des mots de passe faibles et ne sont accompagnés d'aucune

mesure de protection supplémentaire. À travers la mise en œuvre d'un scénario d'attaque par force brute dans un environnement local et strictement pédagogique, il a été possible de démontrer l'efficacité des attaques automatisées lorsque l'application ne limite pas les tentatives de connexion et ne distingue pas un utilisateur légitime d'un script automatisé.

L'analyse du scénario, appuyée par l'utilisation d'outils spécialisés tels que Burp Suite et Hydra, a permis de comprendre le déroulement complet d'une attaque, depuis la phase de reconnaissance jusqu'à l'obtention d'un accès valide. Les résultats obtenus avant la sécurisation de l'application ont clairement montré qu'un accès non autorisé peut être obtenu en un temps très court, soulignant ainsi les risques majeurs liés à l'absence de mécanismes de défense.

La mise en place de contre-mesures simples, notamment la limitation du nombre de tentatives de connexion à trois essais et l'intégration d'un CAPTCHA, a permis de bloquer efficacement l'attaque et de rendre l'automatisation inopérante. La comparaison entre l'application non sécurisée et l'application sécurisée met en évidence l'impact significatif de ces mécanismes, qui renforcent considérablement la sécurité globale du système sans complexifier excessivement l'expérience utilisateur.

En conclusion, ce travail souligne l'importance de l'intégration de bonnes pratiques de sécurité dès la phase de conception des applications web. Il démontre qu'une protection efficace contre les attaques par force brute peut être obtenue grâce à des solutions simples et accessibles, contribuant ainsi à réduire les risques de compromission et à améliorer la fiabilité des systèmes d'authentification.

VII- Annexes

1. Code PHP

```php
<?php

class Login
{
    private $db_connection = null;
    public $errors = array();
    public $messages = array();

    private $max_attempts = 3;
    private $lock_time = 120;

    public function __construct()
    {
        if (session_status() == PHP_SESSION_NONE) {
            session_start();
        }

        // Initialisation compteur brute force
        $_SESSION['login_attempts'] = $_SESSION['login_attempts'] ?? 0;
        $_SESSION['lock_until']     = $_SESSION['lock_until'] ?? 0;

        if (isset($_GET["logout"])) {
            $this->doLogout();
        } elseif (isset($_POST["login"])) {
            $this->doLoginWithPostData();
        }
    }

    private function doLoginWithPostData()
    {
        // Vérifier lockout
        if (time() < $_SESSION['lock_until']) {
            $remaining = $_SESSION['lock_until'] - time();
            $this->errors[] =
                "Too many failed login attempts. Your account is temporarily locked. " .
                "Please wait $remaining seconds before trying again.";
            return;
        }

        // Vérifier si username/password remplis
        if (empty($_POST['user_name']) || empty($_POST['user_password'])) {
            $this->errors[] = "Username or password is empty.";
            return;
        }

```

Figure 1 : Initialisation de la session et mécanisme de limitation des tentatives de connexion

```php
    // Vérification CAPTCHA (obligatoire)
    $captcha_input = $_POST['captcha'] ?? '';
    if ($captcha_input == '') {
        $this->errors[] = "CAPTCHA_REQUIRED: Please solve the CAPTCHA.";
        return;
    }

    if ((int)$captcha_input !== ($_SESSION['captcha_solution'] ?? -1)) {
        $this->errors[] = "WRONG_CAPTCHA: CAPTCHA verification failed.";
        // Ne pas supprimer le captcha pour que l'utilisateur puisse réessayer
        return;
    }

    // Connexion DB
    $this->db_connection = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($this->db_connection->connect_errno) {
        $this->errors[] = "Database connection problem.";
        return;
    }
    $this->db_connection->set_charset("utf8");

    $user_name = $this->db_connection->real_escape_string($_POST['user_name']);
    $sql = "SELECT user_name, user_email, user_password_hash
            FROM users
            WHERE user_name = '$user_name'
            OR user_email = '$user_name'
            LIMIT 1";
    $result = $this->db_connection->query($sql);

    if ($result && $result->num_rows === 1) {
        $user = $result->fetch_object();

        if (password_verify($_POST['user_password'], $user->user_password_hash)) {
            // Login réussi
            $_SESSION['user_name'] = $user->user_name;
            $_SESSION['user_email'] = $user->user_email;
            $_SESSION['user_login_status'] = 1;

            // Reset brute-force et captcha
            $_SESSION['login_attempts'] = 0;
            $_SESSION['lock_until'] = 0;
            unset($_SESSION['captcha_solution'], $_SESSION['captcha_question']);
```

Figure 2 : Vérification CAPTCHA et authentification sécurisée avec base de données

```php
            $this->messages[] = "Login successful.";

        } else {
            $this->registerFailedAttempt();
            $this->errors[] = "Incorrect password.";
        }

    } else {
        $this->registerFailedAttempt();
        $this->errors[] = "User not found.";
    }
}

private function registerFailedAttempt()
{
    $_SESSION['login_attempts']++;

    if ($_SESSION['login_attempts'] >= $this->max_attempts) {
        $_SESSION['lock_until'] = time() + $this->lock_time;
        $_SESSION['login_attempts'] = 0;
        $this->errors[] =
            "Too many failed login attempts. Your account is now locked for " .
            $this->lock_time . " seconds.";
    }
}

public function doLogout()
{
    $_SESSION = array();
    session_destroy();
    $this->messages[] = "You have been logged out.";
}

public function isUserLoggedIn()
{
    return isset($_SESSION['user_login_status']) && $_SESSION['user_login_status'] == 1;
}
}
```

Figure 3 : Gestion des échecs de connexion, verrouillage du compte et déconnexion utilisateur

2. Login page avant sécurité

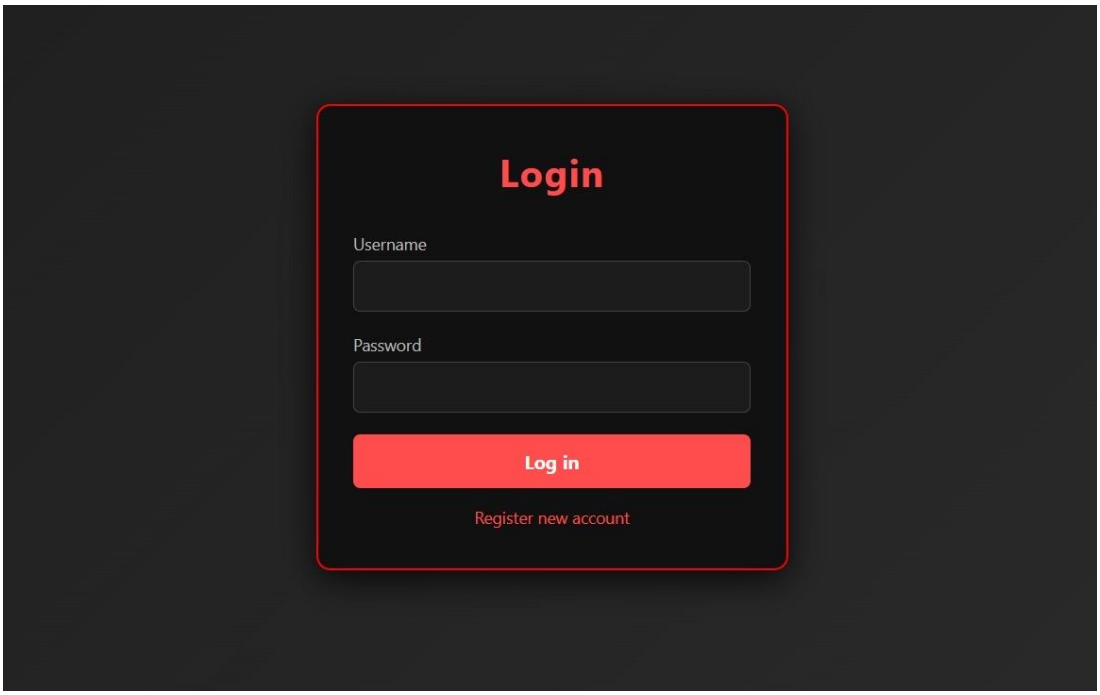


Figure 4 : Interface de connexion sans mécanismes de sécurité

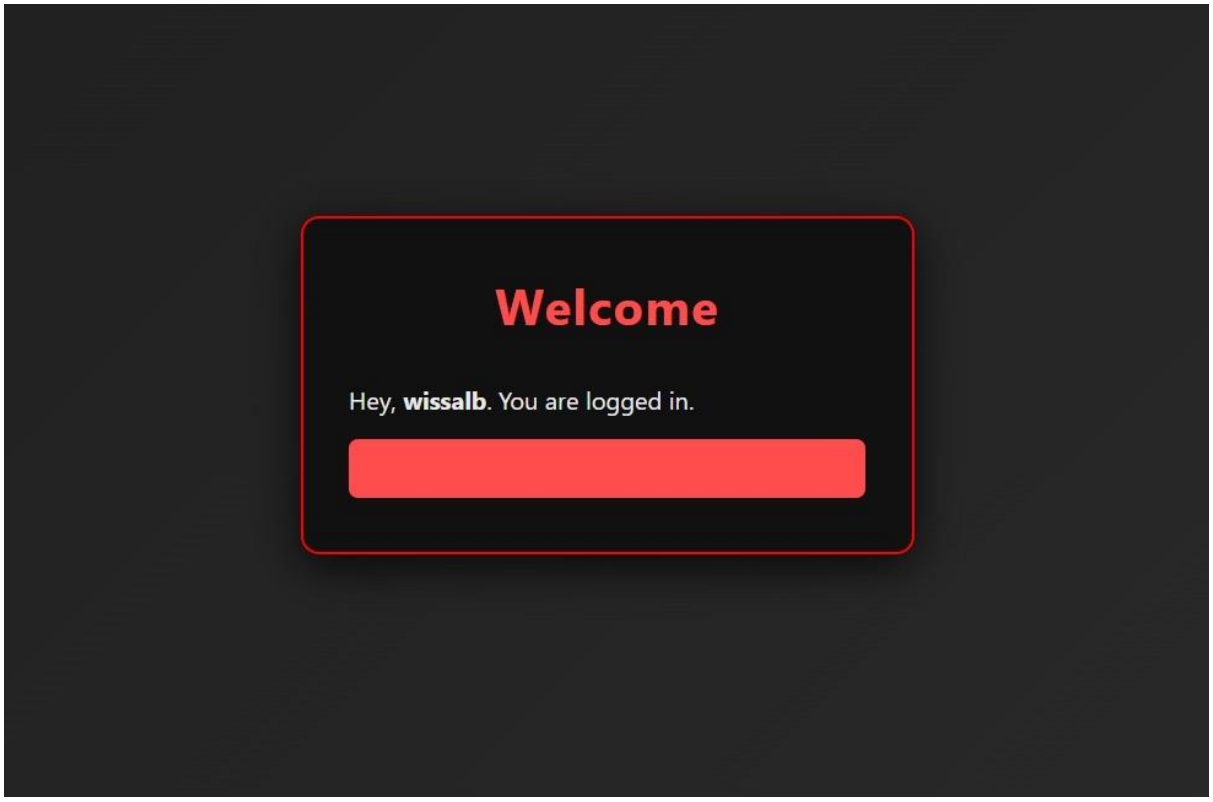


Figure 5 : Connexion réussie sur une application non protégée

3. Login page après sécurité

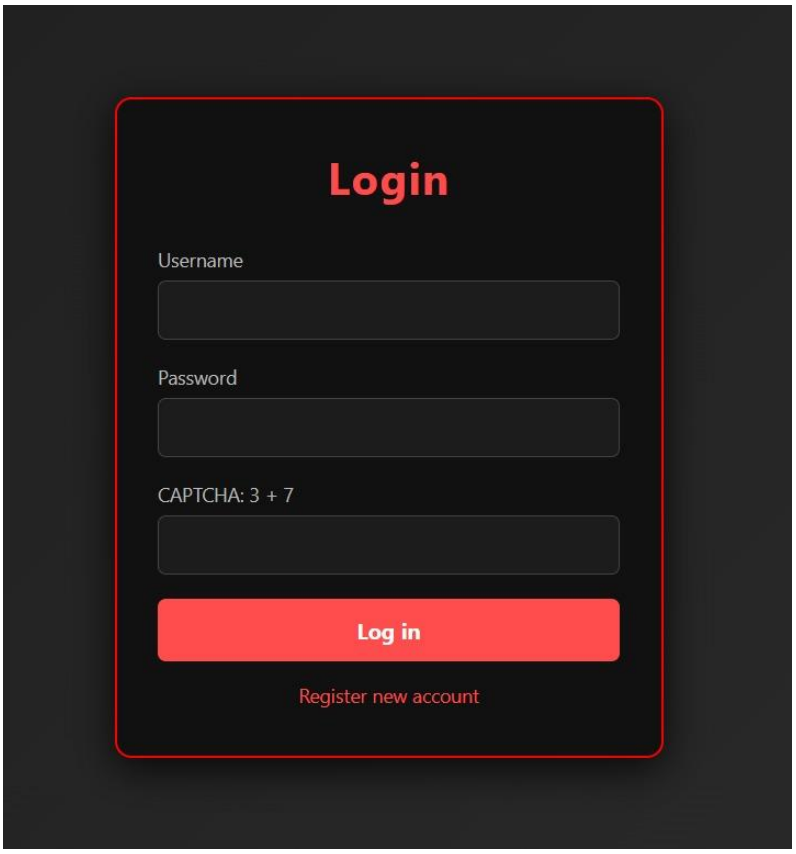


Figure 6 : Interface de connexion avec CAPTCHA et limitation des tentatives

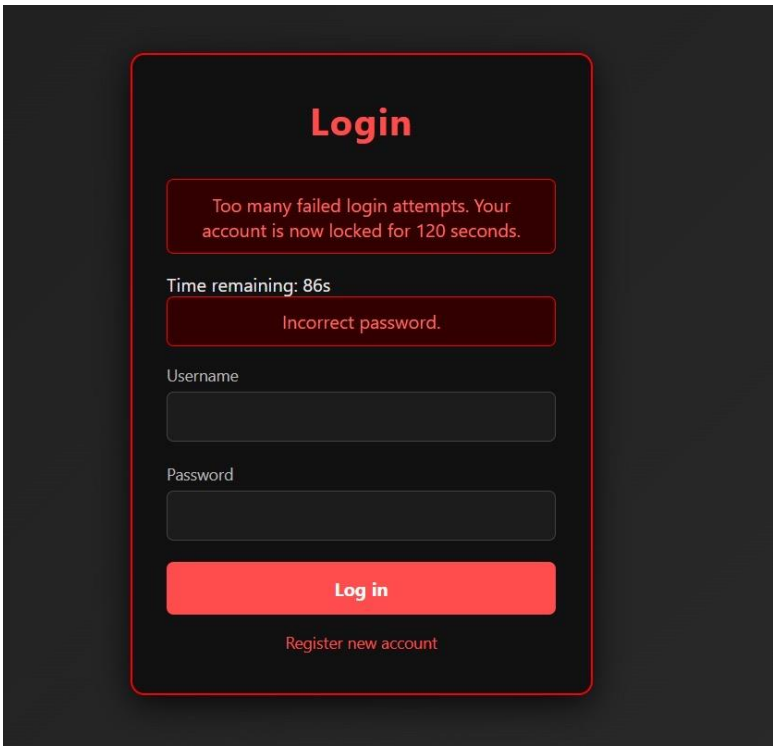


Figure 7 : Verrouillage temporaire du compte après tentatives échouées

4. Outils d'attaque et d'analyse utilisés

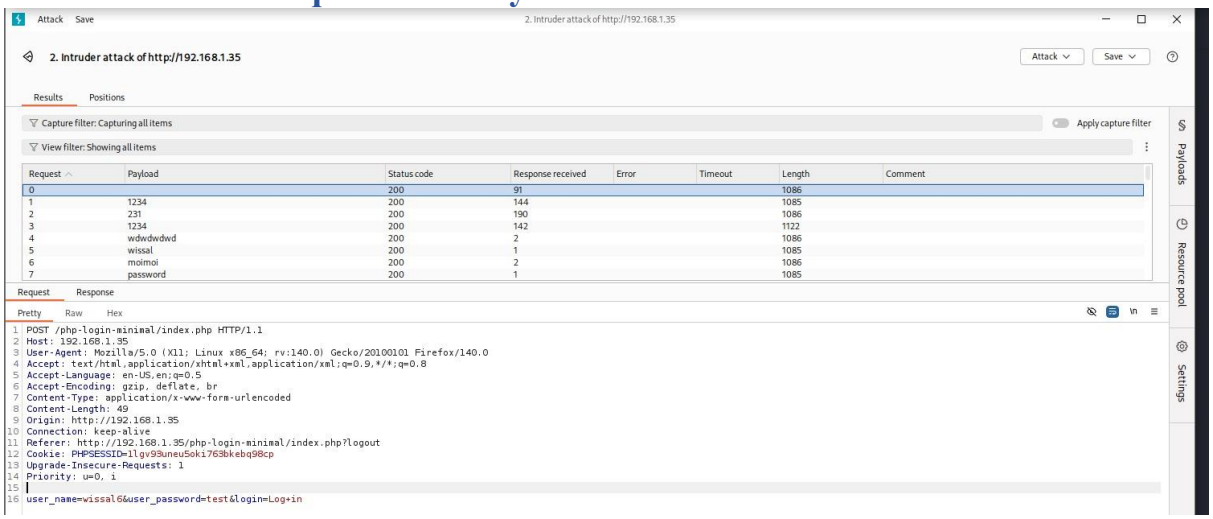


Figure 8 : Burp Suite Intruder avant sécurité

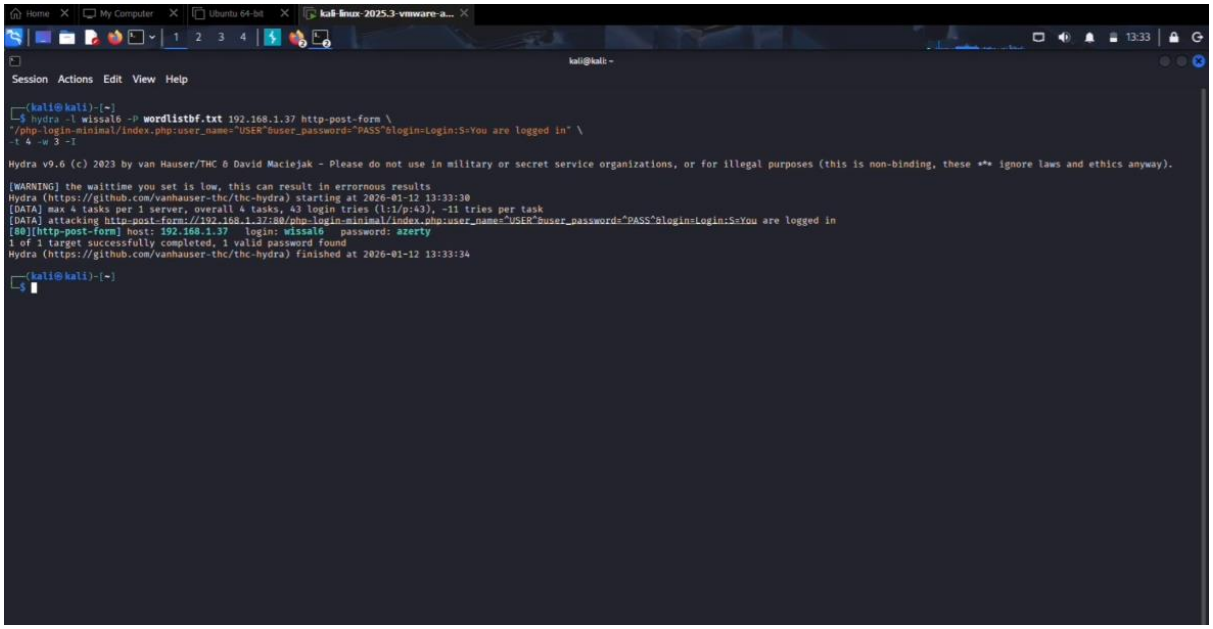


Figure 9 : Hydra avant sécurité

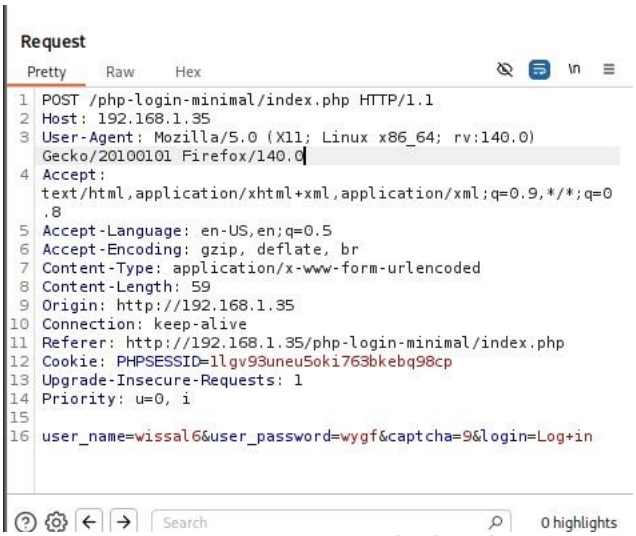


Figure 10 : Burp Suite après sécurité

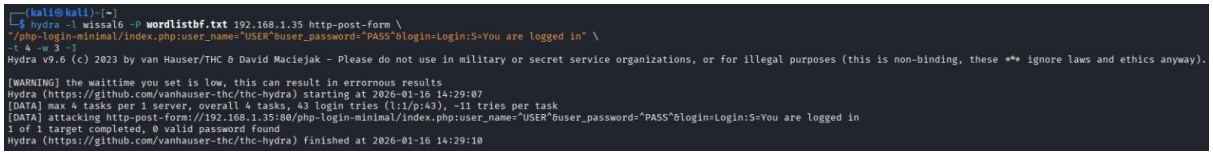


Figure 11 : Hydra après sécurité